

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Downey, Allen B. "Think Bayes 2." *GitHub*, <http://allendowney.github.io/ThinkBayes2/index.html>.

- **Required pages:** Chapter 12, until and including Naïve Bayes Classifier
- **Estimated reading time:** 15 minutes

2. StatQuest with Josh Starmer. "Gaussian Naive Bayes, Clearly Explained!!!" *YouTube*, 3 June 2020, <https://www.youtube.com/watch?v=H3EjCKtIVog&list=PLSV3ndw2r1qUojFqv5Vt6KNIRmVN7Zic6&index=10>.

- **Required pages:** Whole video
- **Estimated reading time:** 9 minutes

BAYESIAN STATISTICS IN CLASSIFICATION

Reading Time	60 minutes
Prior Knowledge	Basic probability theory, Financial markets
Keywords	Bayesian statistics, Likelihood, Posterior, Prior

In the previous module, we looked at unsupervised machine learning. We will now use the explanatory variables to predict a target variable. This will be the first supervised machine learning algorithm that we'll cover.

1.The Bayesian Framework

We often want to find the probability that an observation will take on a certain class value, that is a success/failure or buy/hold, as an example of a binary classification problem. These events are known as a collective exhaustive set of events, in which there are only two possible outcomes, say A and $\bar{A}$, $P(A) + P(\bar{A}) = 1$. Therefore, if we know the probability of one outcome, we can obtain the probability of the other by simply taking the complement, i.e., $P(A) = 1 - P(\bar{A})$.

A probability of an event can depend on whether another event has taken place. For instance, we might consider the probability of a student passing an exam given that the student has studied for over 40 hours. In this case, the probability of passing the exam should be higher than if the student had studied for just four hours. If we denote the events:

- S - The event that the student passes the exam
- A - The event that the student prepares for the exam

We can write the statistical notation of $P(S|A)$ as the probability of the student passing the exam given they have prepared for it. This is known as **conditional probability** and is the building block of Bayesian statistics.

The **joint probability** is the probability of events occurring together, i.e., $P(A \cap B)$ is the probability of event A and B occurring. From the example above, $P(A \cap S)$ would be the probability that the student prepares for the exam and passes, i.e., the joint probability of A and S .

1.1 Conditional Probability

For any two events A and B , the conditional probability rule is given by, $P(A|B) = \frac{P(A \cap B)}{P(B)}$ (1.1).

Let's use the above rule in an example. A sample is taken of employees in the mining and finance sector who hold positions as either manager or director. Table 1.1 shows the sample in a contingency table. Based on the sample, suppose we wanted to know the probability of an employee being a manager given they are in the finance sector. Using (1.1), we have

$$\begin{aligned} P(\text{Management}|\text{Finance}) &= \frac{P(\text{Management} \cap \text{Finance})}{P(\text{Finance})} \\ &= \frac{700/1600}{900/1600} \\ &= 0.78 \end{aligned}$$

Table 1

	Mining	Finance	TOTAL
Manager	580	700	1,280
Director	120	200	320
TOTAL	700	900	1,600

We now have an understanding of the basic probability theory needed for the Bayesian statistics approach. We will start by looking at Bayes's theorem and the Naive Bayes classifier that is based on the theorem.

Let us look at a video that will help us put this theory of probability into practice:

04:24

[Access video transcript here](#)

1.2 Bayes Theorem

Rearranging (1.1) we have the joint probability given by $P(A \cap B) = P(A|B)P(B)$.

Keep in mind that the event of A and B are the same as B and A ; therefore, the above can also be written as $P(A \cap B) = P(B|A) \cdot P(A)$ (1.2).

Substituting (1.2) into (1.1), we obtain $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$ (1.3).

Equation (1.3) is known as Bayes theorem, which forms the basis for most algorithms using the Bayesian approach. Equation (1.3) can be extended to more than just one conditional variable. In other words, instead of just B , we can have $X_1, X_2, \dots, X_n$. We write X instead to keep it consistent with the notation of explanatory variables, and we will use Y instead of A since the target or response variable is generally written as such. We will need this notation to explain the Naive Bayes classifier methodology. Now that we've seen the Bayes theorem, let's use this to build a classifier.

1.3 Naive Bayes

Suppose we have a target variable that takes on class values. With Y taking on a specific class value, say k , equation (1.3) can be written as (1.4):

$$P(Y = k|X_1, X_2, \dots, X_n) = \frac{P(X_1|Y = k)P(X_2|Y = k) \cdots P(X_n|Y = k)P(Y = k)}{P(X_1)P(X_2) \cdots P(X_n)}$$

It is important to note that equation (1.4) is true provided the explanatory variables $X_1, X_2, \dots, X_n$ are conditionally independent of Y , i.e. $P(X_1, X_2, X_n|Y = k) = \prod_{i=1}^n P(X_i|Y = k)$. This is the "naive" assumption made for the classifier and hence the name. Naive Bayes assumes conditional independence whereas Bayes's theorem does not. The denominator can also be written as,

$$P(X_1 \cdots X_n) = \sum_j P(Y = j) \prod_{i=1}^n P(X_i|Y = j)$$

by summing over all possible classes j of the target variable. This is a property of marginalization. There are specific names given to the terms in (1.4), namely:

- $P(X_1|Y = k) \cdot P(X_2|Y = k) \dots P(X_n|Y = k) = \textit{likelihood}$
- $P(Y = k) = \textit{Prior}$
- $P(X_1 \cdot X_2 \dots X_n) = \textit{Evidence}$
- $P(Y = k|X_1, X_2, \dots, X_n) = \textit{Posterior}$

so we have (1.5):

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

To help you conceptualize this, please watch the following video.

03:44

[Access video transcript here](#)

1.4 Naive Bayes Example

Let us look at an example to understand the classifier. Table 1.2 contains credit risk data, specifically credit card defaulters and non-defaulters on their loans. There are two explanatory variables namely X_1 and X_2 , which are industry sector of employment and length of employment being more than 3 years or not. Determining the probability of defaulting on a loan is an important focus in the banking industry.

Table 2

	X_1	X_1	X_1	X_2	X_2	
	Mining	Finance	Agriculture	Employed > 3 yrs	Employed 3 yrs or less	TOTAL
Default (y=1)	250	600	150	300	700	1000
Non-default (y=0)	1250	6000	1750	1500	7500	9000
TOTAL	1500	6600	1900	1800	8200	10000

If we wanted the likelihood for an employee in the mining sector given they defaulted, we would use (1.1) to obtain (1.6):

$$\begin{aligned} P(X_1 = \textit{Mining}|Y = 1) &= \frac{250}{1000} \\ &= 0.25 \end{aligned}$$

and similarly, the likelihood of being employed for 3 years or less given the employee defaulted is

$$\begin{aligned} P(X_2 = \textit{Employed3yrsunder}|Y = 1) &= \frac{700}{1000} \\ &= 0.7. \end{aligned}$$

We could now determine the Naive Bayes probability of defaulting given the employee is in the mining sector and has been employed for three years or less using (1.4). For simplicity, let M = Mining and Eu = Employed 3 years or less.

$$\begin{aligned} P(Y = 1 | X_1 = M, X_2 = Eu) &= \frac{P(X_1 = M|Y = 1) \cdot P(X_2 = Eu|Y = 1) \cdot P(Y = 1)}{P(X_1 = M) \cdot P(X_2 = Eu)} \\ &= \frac{(0.25)(0.7)(1000/10000)}{(9000/10000)(1250/9000)(7500/9000) + (1000/10000)(250/1000)(700/1000)} = 0.14. \end{aligned}$$

Similarly, the $P(Y = 0|X_1 = \textit{Mining}, X_2 = \textit{Employed3yrsorless}) = 0.85$. The classifier would assign the client as a non-defaulter since they have a higher probability of non-default. A useful reference for more details on Bayesian analysis can be found in chapter 12 of the Downey reading.

We have shown how the Naive Bayes classifier works and the assumptions made. The classifier can also be extended to continuous predictor variables with the assumption of normality. We've also included a [link to another helpful video](#).

To consolidate the information presented above watch the following video:

07:14

[Access video transcript here](#)

2. Conclusion

In this lesson, we presented the methodology behind Bayesian statistics and the workings of the Naive Bayes classifier. We saw this probability approach used to address classification problems by using the prior and likelihood probabilities to calculate the chance of being classified as a particular class. In a simplified example with few predictors is used to grasp the concept. The following lesson will implement this tool in a practical financial problem with more predictors. Therefore, we'll incorporate Python to assist with the heavy lifting.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Coqueret Guillaume and Tony Guida. *Machine Learning for Factor Investing*. CRC Press LLC 2020.

<https://www.mlfactor.com/bayes.html>

- **Required pages:** Section 9.4
- **Estimated reading time:** 30 minutes

BAYESIAN STATISTICS IN PRACTICE

Reading Time	15 minutes
Prior Knowledge	Naïve Bayes classifier methodology, likelihood, prior.
Keywords	Gaussian Distributions, Model fitting, Likelihood, Prior distribution

In the previous lesson, we looked at how Bayesian statistics can be used to assign the probability of an outcome occurring. We looked at categorical features and will now look at continuous features to predict the future one-month return of a group of stocks exceeding the median.

1. Naïve Bayes in Practice

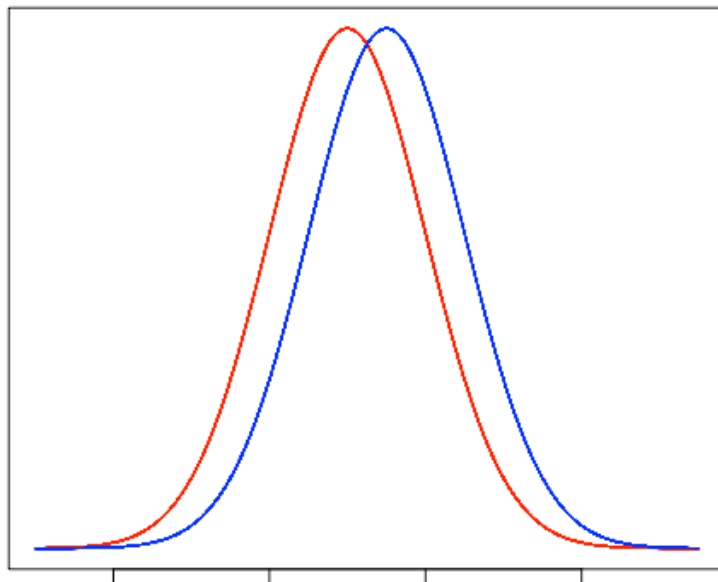
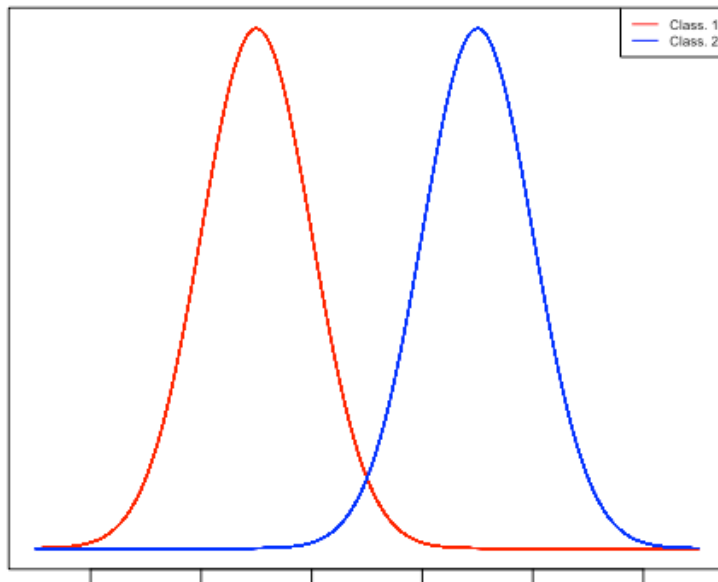
Recall from Module 3, Lesson 1, that posterior distribution can be expressed as a product of the likelihoods of the predictors given the class and the prior distributions of the class, as in (1.4). This is based on the assumption of conditional independence. This is true for predictors that are numeric and continuous as well since these predictors also have likelihood distributions given a specific target class.

$$P(Y = k|X_1, X_2, \dots, X_n) = \frac{P(X_1|Y = k)P(X_2|Y = k) \cdots P(X_n|Y = k)P(Y = k)}{\sum_j P(Y = j) \prod_{i=1}^n P(X_i|Y = j)}$$

(2.1)

Let's consider a case where the likelihood distribution of the predictor follows a Gaussian distribution for each class. The predictor would be a strong explanatory variable, should the distributions differ. A strong predictor, which is often what we aim for, has very little overlap between the likelihoods for each class, thus differentiating between classes. This is shown in figure 1 for both a strong and weak predictor. The top plot shows the likelihood for a strong predictor given the target class as there is little overlap between the distributions (the two curves). The bottom plot shows a weaker predictor with significantly more overlap. All rights reserved WQU WorldQuant University QQQQ

Fig 1: Strong vs. Weak Predictor Likelihoods



To put these ideas into perspective, let us look at an example where we apply the Naïve Bayes classifier to a stock returns problem.

In this section, we use data from Coqueret and Guida (chapter 9, section 9.4), specifically the future one-month returns data for a collection of stocks and financial data. This data can be accessed via <https://github.com/shokru/mlfactor.github.io/tree/master/material>. For illustrative purposes, we take a subset of stocks and features to predict the future one-month price being above the median, thus a classification problem. The features we look into are:

- *Mkt_Cap_12M_Usd* - Size
- *Pb* - Price to book
- *Vol1Y_Usd* - For share turnover
- *Return_On_Capital*
- *Roe* - Return on equity
- *Pe* - Price to Earnings ratio

We perform the analysis using the Python statistical software. Let us load the necessary libraries and import the data. We'll work with data from the year 2000 to 2018 and only 50 stock IDs.

```
In [1]: import numpy as np
import pandas as pd

df_50 = pd.read_csv("mlfac_dat.csv")

# consider data from 2000 to 2018

df_50 = df_50[(df_50["date"] > "1999-12-31") & (df_50["date"] < "2019-01-01")]

# Filter on the first 50 stock ids

df_50 = df_50[df_50["stock_id"] <= 50]
```

The objective is to predict whether the future one-month returns are above the median. Hence, we can simply flag instances when it is above the median as 1 and 0 otherwise.

```
In [2]: # Created a class variable for 1-Month returns if > median

df_50["R1M_Usd_C"] = np.where(df_50["R1M_Usd"] > df_50["R1M_Usd"].median(), 1, 0)

# Make the response variable into integer format

df_50["R1M_Usd_C"] = df_50["R1M_Usd_C"].astype(int)
```

Specify the predictors or features used in the classifier. These features have been transformed into variables that follow a uniform distribution. In practice, we can hope for Gaussian-distributed variables. Therefore, we will transform each feature using the Box-Muller transformation (Box and Muller 610).

```
In [3]: # create a copy of data to store Gaussian converted data

df50_norm = df_50.copy()

# Features of interest
feature_cols = ["Mkt_Cap_12M_Usd", "Pb", "Vol1Y_Usd", "Return_On_Capital", "Roe", "Pe"]

# loop through Box-Muller for each data point for each predictor
for col in feature_cols:
    for i in np.arange(len(df_50)):
        # 1. Use U1 as current data value. Generate U2, which is Unif(0, 1)

        u1s, u2s = (
            df_50.loc[i, col],
            np.random.uniform(low=0.0, high=1.0, size=1)[0],
        ) # X.loc[i, col]

        # 2. Transform U1 to s

        ss = -np.log(u1s)

        # 3. Transform U2 to theta

        thetas = 2 * np.pi * u2s

        # 4. Convert s to r

        rs = np.sqrt(2 * ss)

        # 5. Calculate x and y from r and theta

        xs, ys = rs * np.cos(thetas), rs * np.sin(thetas)

        # 6. Store only one of the Gaussian derived values

        df50_norm.loc[i, col] = ys
```

It is worth looking at the likelihoods given the target class to gauge how strong or weak we can expect the predictors to be. We do this using the code below:

```
In [4]: import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(2, 3, figsize=(18, 10), sharey=True)

i = 0

sns.kdeplot(ax=axes[0, i], data=df50_norm, x=feature_cols[i], hue="R1M_Usd_C")

i = 1

sns.kdeplot(ax=axes[0, i], data=df50_norm, x=feature_cols[i], hue="R1M_Usd_C")

i = 2

sns.kdeplot(ax=axes[0, i], data=df50_norm, x=feature_cols[i], hue="R1M_Usd_C")

i = 0

sns.kdeplot(ax=axes[1, i], data=df50_norm, x=feature_cols[i + 3], hue="R1M_Usd_C")

i = 1

sns.kdeplot(ax=axes[1, i], data=df50_norm, x=feature_cols[i + 3], hue="R1M_Usd_C")

i = 2

sns.kdeplot(ax=axes[1, i], data=df50_norm, x=feature_cols[i + 3], hue="R1M_Usd_C")
```

Out[4]: <Axes: xlabel='Pe', ylabel='Density'>

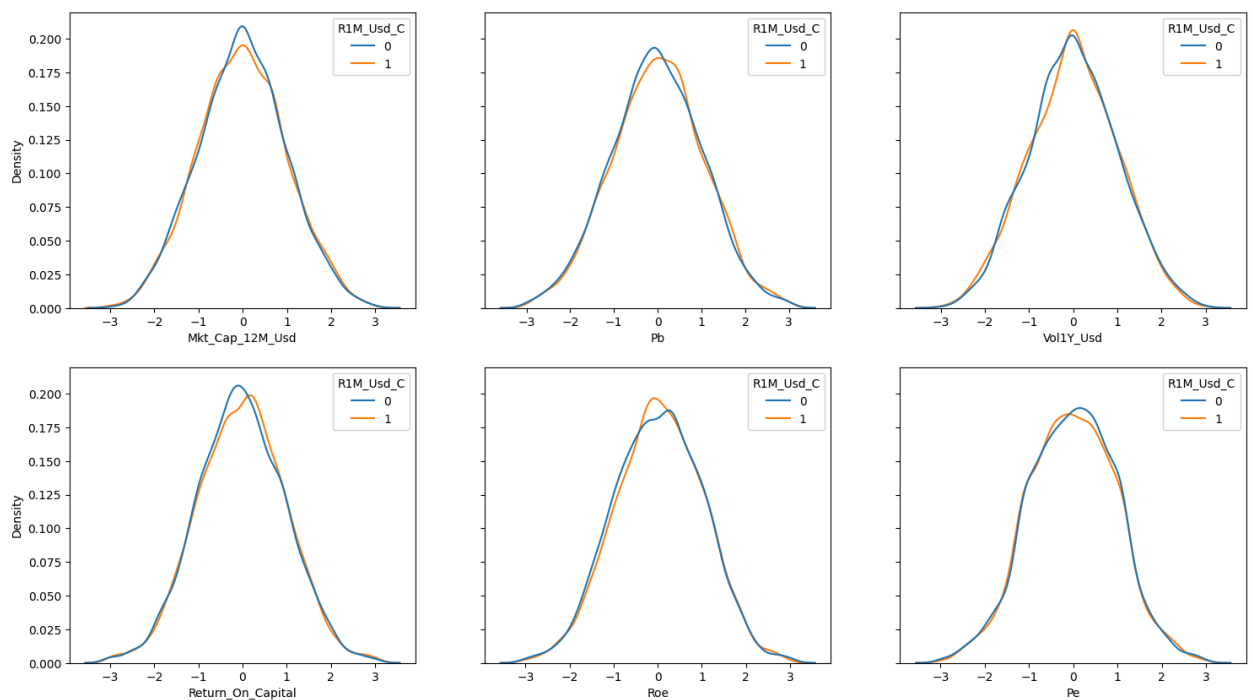


Fig. 2: Likelihood for All 6 Features Given the Respective Target Class

Looking at the output in figure 2 and comparing it to figure 1, we can see the predictive power of the features are quite weak considering the significant overlap in the likelihoods. We can therefore expect poor model performance of the classifier.

We then load the library needed to split the dataset into a training and testing dataset to train and evaluate the classifier, respectively. An 80/20 train/test split is used. We consider only the features mentioned above.

```
In [5]: # split dataset in features and target variable

X = df50_norm[feature_cols] # Transformed Features

y = df50_norm["R1M_Usd_C"] # Target variable

# Split dataset into training set and test set 80/20

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Let us fit the Naïve Bayes classifier on the training set and obtain predictions on the unseen test set.

```
In [6]: # initialize and fit classifier to training set
from sklearn.naive_bayes import GaussianNB

classifier = GaussianNB()

classifier.fit(X_train, y_train)

# get the predictions

y_pred = classifier.predict(X_test)

# Extract the probabilities of Target Class = 1.

y_predprobs = classifier.predict_proba(X_test)

probs = y_predprobs[:, 1]

# print the accuracy

from sklearn import metrics

# Model Accuracy, how often is the classifier correct?

print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

Accuracy: 0.5042091271599468

The ROC curve and AUC can be obtained using the code below. It is useful to also show a no-skill or random guess classifier in the plot to compare to the Naïve Bayes classifier.

```

In [7]: # plot the roc curve for the model
from sklearn.metrics import roc_auc_score, roc_curve

# generate a no skill or random guess prediction
ns_probs = [0 for _ in range(len(y_test))]

# calculate roc curves
ns_fpr, ns_tpr, _ = roc_curve(y_test, ns_probs)

# calculate scores
ns_auc = roc_auc_score(y_test, ns_probs) # random guess
tree_auc = roc_auc_score(y_test, probs) # tree classifier

# summarize scores
print("No Skill: ROC AUC=%.3f" % (ns_auc))
print("Classifier: ROC AUC=%.3f" % (tree_auc))

plt.plot(ns_fpr, ns_tpr, linestyle="--", linewidth=4, label="No Skill")

plt.plot(
    ns_fpr,
    ns_tpr,
    linewidth=2,
    label="NB Classifier",
)

# axis labels
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

# show the legend
plt.legend()

# show the plot
plt.show()

```

```

No Skill: ROC AUC=0.500
Classifier: ROC AUC=0.508

```

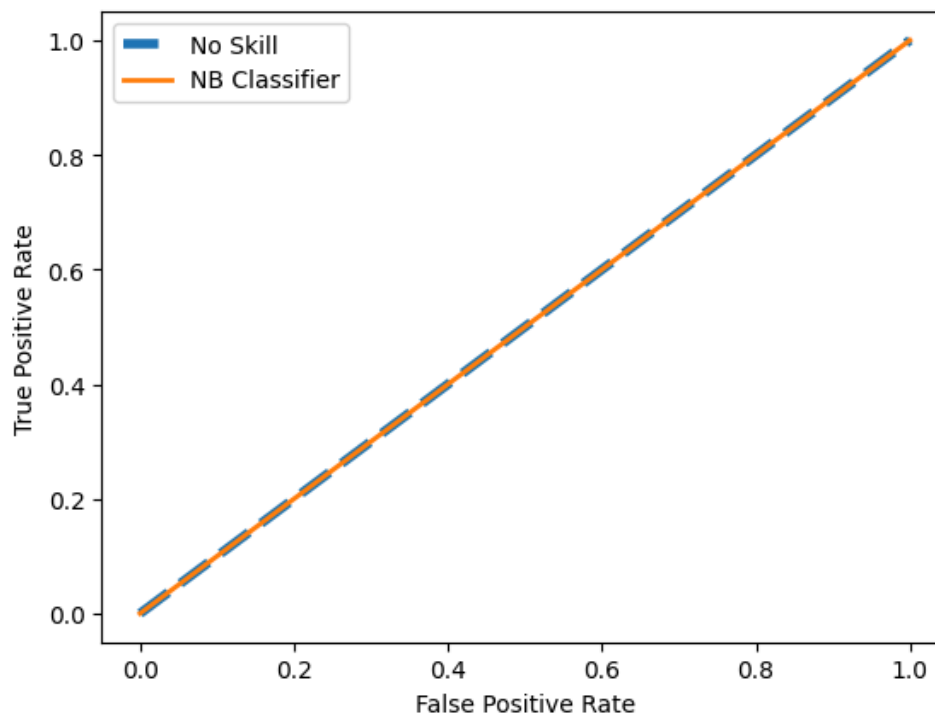


Fig. 3: ROC Curve of Naïve Bayes Classifier Performance along with a Random Guess or No-Skill Model

The corresponding ROC curve is shown in figure 3. The AUC for this model is ~50%, thus confirming our suspicion of its weak predictive power.

2. Conclusion

The example covered in this lesson helps us understand the Naïve Bayes classifier by seeing it in practice. The model performance, however, is not as good as we would hope. This lesson focused on showing how to apply Naïve Bayes to a real-world classification problem using continuous predictors and how Naïve Bayes can easily be applied to categorical variables. To obtain a better model, stronger predictors are needed, i.e., explanatory variables that have different likelihood distributions for the target classes. Explanatory data analysis, feature selection, and engineering cannot be understated when dealing with modeling.

In the next section, we will look at tree-based algorithms, another modeling technique. The Naïve Bayes model assumes the features are conditionally independent of the target variable; however, tree-based algorithms do not require any assumptions about the underlying distribution. Both Naïve Bayes and trees do not require feature scaling, although tree-based algorithms are not sensitive to outliers. The prediction approach of trees are to place decision boundaries in the data whereas Naïve Bayes aims to model how the data were generated. These are known as discriminative and generative models, respectively. We will look at trees in depth in the next lesson.

References

- Box, G. E. P., and Mervin E. Muller. "A Note on the Generation of Random Normal Deviates." *The Annals of Mathematical Statistics*, vol. 29, no. 2, 1958, pp. 610–11, <https://doi.org/10.1214/aoms/1177706645>.
- Coqueret, Guillaume, and Tony Guida. *Machine Learning for Factor Investing: R Version*. Financial Mathematics Series. Chapman and Hall/CRC, 2020.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. James, Gareth, et al. *An Introduction to Statistical Learning with Applications in R*. 2nd ed., Springer, 2021.

- **Required pages:** Chapter 8, pp. 327–348
- **Estimated reading time:** 45 minutes

Note: This book is not open access; however, the publisher and authors have made a downloadable copy available online. Due to copyright issues, we cannot provide you with a direct link, but if you google the book's title, you should be able to find and download a copy of it.

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

Optional Readings

1. Murphy, Kevin Patrick. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022,
<https://probml.github.io/pml-book/book1.html>.

- **Required pages:** pp. 627–632

TREES

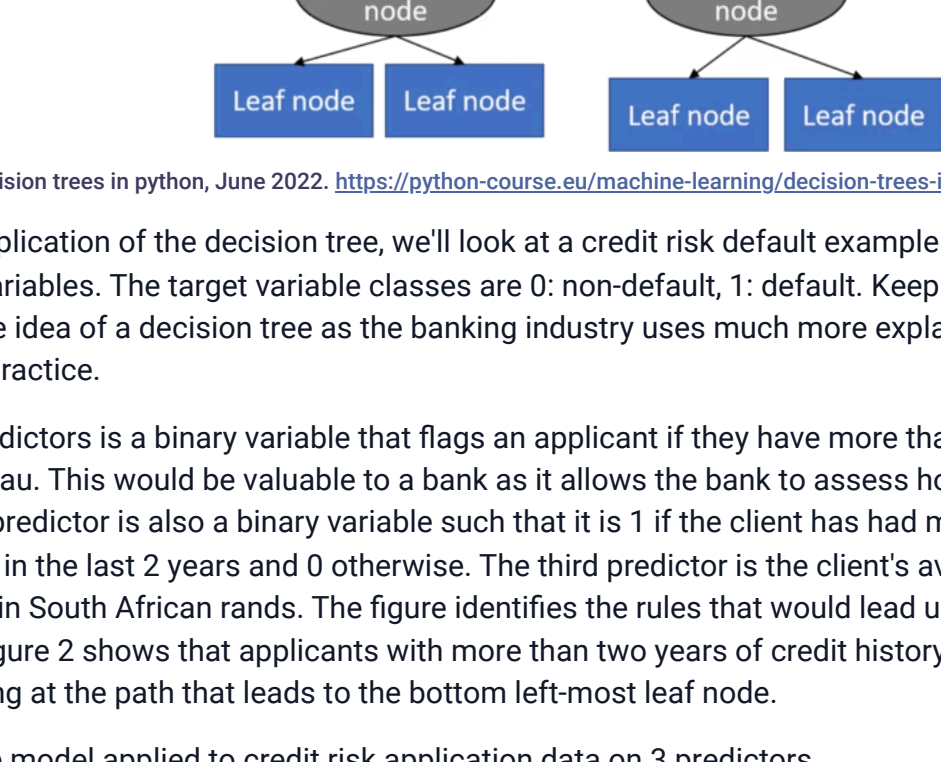
Reading Time	90 minutes
Prior Knowledge	Logistic regression
Keywords	Gini Impurity, Nodes, Homogeneous nodes, Boosting, ROC Curve

We have looked at one of the tools for classification, namely *Naïve Bayes classification*. This method builds on the framework of *Bayesian statistics* and makes use of a prior and *likelihood distribution*. These distributions are based on the data we train the model on, and we make the *naïve assumption of independence between predictors*. Oftentimes, this is not the case with real-world problems. Let us now look at an alternative method that does not make such an assumption and is widely used in practice.

1. Classification Trees

To understand tree-based models, we need to understand the basic building blocks, which are decision trees. The components of decision trees are shown in figure 1. These are nodes that have specific names based on their position in the flow diagram. By following the flow from top to bottom, we initially have the root, interior, and finally the leaf nodes. The splits into left/right directions are called **branches**.

Figure 1: The Basic Components and Structure of a Decision Tree.

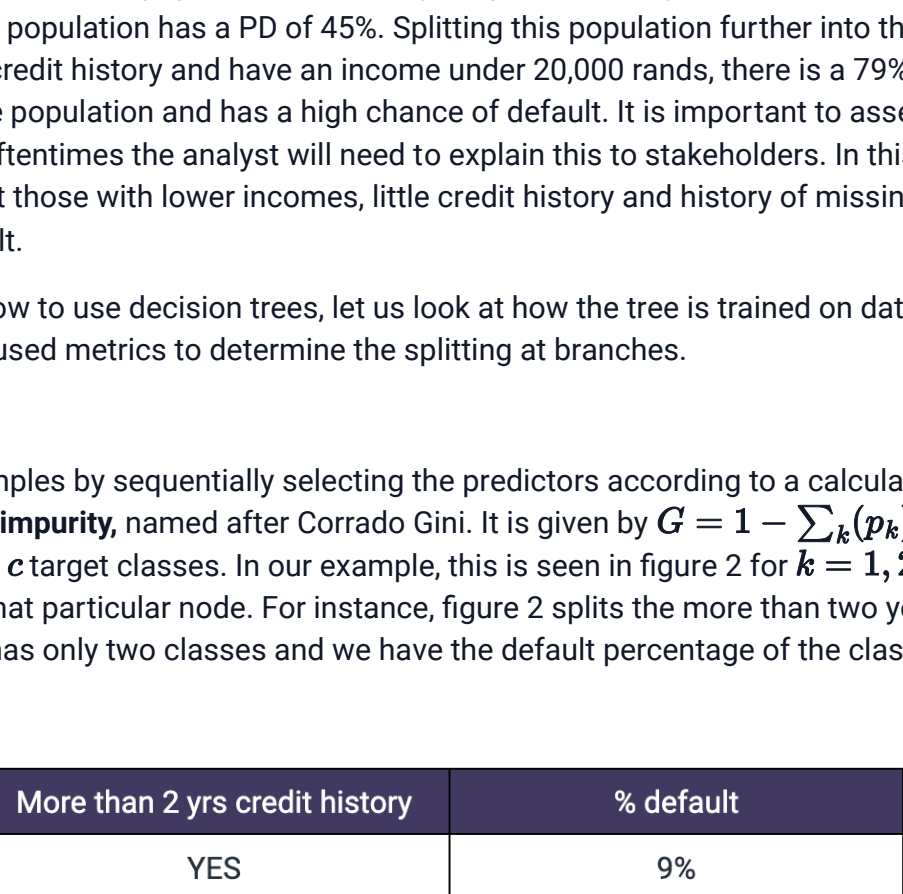


Source: python-course.edu, Decision trees in python, June 2022. <https://python-course.eu/machine-learning/decision-trees-in-python.php>.)

To conceptualize the application of the decision tree, we'll look at a credit risk default example for a bank loan based on just three explanatory variables. The target variable classes are 0: non-default, 1: default. Keep in mind that this example is to convey the idea of a decision tree as the banking industry uses much more explanatory variables/predictors in practice.

The first of the three predictors is a binary variable that flags an applicant if they have more than two years of credit history. This would be valuable to a bank as it allows the bank to assess how the client performs financially. The second predictor is also a binary variable such that it is 1 if the client has had more than 1 missed payment for other loans in the last 2 years and 0 otherwise. The third predictor is the client's available monthly income. Assume the currency is in South African rand. The figure identifies the rules that would lead us to the particular target classes. For instance, figure 2 shows that applicants with more than two years of credit history have a 9% probability of defaulting (PD) by looking at the path that leads to the bottom left-most leaf node.

Figure 2: A decision tree model applied to credit risk application data on 3 predictors.



This population makes up 60% of the population. Following the path to the right, we look at those with at most two years of credit history. This population has a PD of 45%. Splitting this population further into those that had more than 1 arrears in their two-year credit history and have an income under 20,000 rand, there is a 79% chance of default. This group makes up 20% of the population and has a high chance of default. It is important to assess the logic and the sensibility of the rules as oftentimes the analyst will need to explain this to stakeholders. In this example, the data confirms our suspicion that those with lower incomes, little credit history and history of missing more than one payment are likely to default.

Now that we understand how to use decision trees, let us look at how the tree is trained on data. To do this, we first examine some commonly used metrics to determine the splitting at branches.

2. Gini Impurity

Decision trees split the samples by sequentially selecting the predictors according to a calculated measure. The measure we look at is **Gini impurity**, named after Corrado Gini. It is given by $G = 1 - \sum_k (p_k)^2$ (3.1) where the sum is over all possible $k = 1, \dots, c$ target classes. In our example, this is seen in figure 2 for $k = 1, 2$. The terms p_k are the proportion of the class in that particular node. For instance, figure 2 splits the more than two years of credit history variable first. This feature has only two classes and we have the default percentage of the classes in table 1.

Table 1

More than 2 yrs credit history	% default
YES	9%
NO	91%

We therefore have from (3.1),

$$G = 1 - [(0.09^2) + (0.91^2)] = 0.164$$

The idea behind Gini impurity is to obtain as pure a node as possible, thereby minimizing the risk of misclassification. We want as low a Gini impurity as possible to achieve this. You can see that the left-most leaf node has a very low percentage of default population. Similarly, the inner node for those with ≤ 2 yrs credit history has a Gini impurity of 0.495. The last two leaf nodes on the right have the following Gini impurity:

Table 2

Leaf	Gini Impurity
≤ 2 yrs history, more than 1 arrears, at least R20k income	18%
≤ 2 yrs history, more than 1 arrears, and < R20k income	33%

The variable at the root node is chosen to produce the purest node, i.e., the lowest Gini impurity. The decision tree has to therefore split on all predictors to determine this.

Another commonly used measure for splitting a tree is entropy. Entropy or Gini impurity is used to maximize **information gain**. The reader is referred to Hull (1933) should they be interested in understanding this. We've seen trees in a classification problem, but keep in mind that they can also be used for a continuous target variable. These are regression problems, which we cover next.

For an application of decision trees with some mock data, this video should clarify the concepts discussed above:

12:55

[Access video transcript here](#)

3. Regression Trees

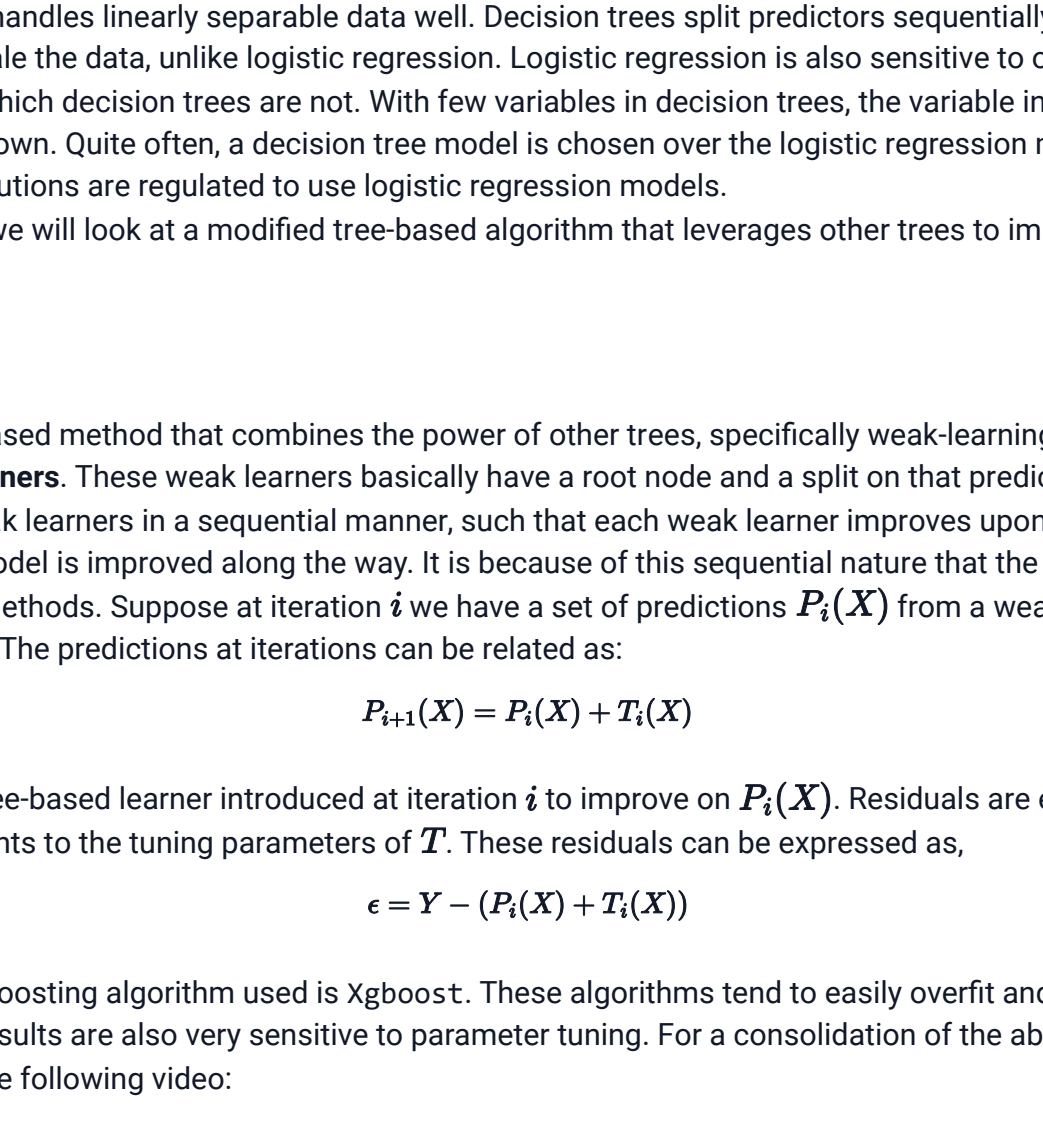
Thus far, we have referred to only classification problems; however, we may in practice be faced with a continuous target variable. In this instance, we would be interested in predicting the amount or value in a financial problem. In this type of problem, a regression tree would be applicable. The difference in the chart as compared to figure 2 is that the leaf nodes would show the mean value of the target variable. The data is split for each predictor at every point, and the split with the lowest variation between the predicted and actual target values is chosen as the splitting point. The measure of variation is the sum of squares, which should be familiar from linear regression. For any predictor i we have (3.2):

$$SSE = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

with $\bar{y}_i$ the predicted value and $\bar{y}_1$ and $\bar{y}_2$ the mean value of the left and right side of the split. The split that produces the lowest SSE would be chosen. Trees, like other machine learning algorithms, are susceptible to overfitting; however, there are ways to minimize this. One of them is called pruning.

Pruning aims to reduce the size of a decision tree, thus reducing overfitting. This is done by reducing sections of the tree that are non-critical. We ideally want the tree to generalize unseen samples well. One of the pruning strategies involves producing a tree with small samples in the leaf nodes, thereby reducing the maximum depth of the tree and eliminating those overfitted leaves. This is an example of post-pruning. With pre-pruning, we specify parameters to prevent leaves with too few samples; an example would be maximum depth or minimum Gini. This early stoppage can be efficient as there are fewer computations. Figure 3 illustrates an unpruned vs. pruned tree.

Figure 3: Unpruned vs. Pruned Tree



One of the most simplistic predictive classification algorithms is logistic regression, and it is worth mentioning the comparisons to decision trees.

4. Logistic Regression vs. Decision Trees

Both methods deal with classification problems and are easily interpreted. Decision trees can split the data non-linearly; hence, the data does not have to be linearly separable.

Figure 4: Non-Linearly Separable Data



Figure 4 illustrates the non-linearly separable data of which a decision tree is more effective at class predicting. Logistic regression handles linearly separable data well. Decision trees split predictors sequentially; therefore, it is also not necessary to scale the data, unlike logistic regression. Logistic regression is also sensitive to outliers and skewness of data, which decision trees are not. With few variables in decision trees, the variable interaction is easily interpretable and shown. Quite often, a decision tree model is chosen over the logistic regression model; however, some financial institutions are regulated to use logistic regression models.

In the next section, we will look at a modified tree-based algorithm that leverages other trees to improve predictive performance.

5. Boosting

Boosting is a tree-based method that combines the power of other trees, specifically weak-learning trees, which we refer to as **weak learners**. These weak learners basically have a root node and a split on that predictor. The process combines these weak learners in a sequential manner, such that each weak learner improves upon the previous learner, meaning the final model is improved along the way. It is because of this sequential nature that the training takes longer than other parallel methods. Suppose at iteration i we have a set of predictions $P_i(X)$ from a weak learner based on a set of predictors X . The predictions at iterations can be related as:

$$P_{i+1}(X) = P_i(X) + T_i(X)$$

where $T_i(X)$ is a tree-based learner introduced at iteration i to improve on $P_i(X)$. Residuals are examined to determine adjustments to the tuning parameters of T . These residuals can be expressed as,

$$\epsilon = Y - (P_i(X) + T_i(X))$$

A popular gradient boosting algorithm used is Xgboost. These algorithms tend to easily overfit and are sensitive to noise in the data. Results are also very sensitive to parameter tuning. For a consolidation of the above concepts with mock data, watch the following video:

05:28

[Access video transcript here](#)

Thus far, we have looked at accuracy for assessing model performance. In the next section, we will examine a commonly used metric and diagram to assess model performance.

6. Receiver Operating Curve

We'll discuss a measure used for classification models, specifically the receiver operating curve (ROC) and area under the curve (AUC). These measures actually stem from the Gini coefficient and Lorentz curve, which were initially used to show income inequality in countries. We will not explore the Gini coefficient as we will stick to just the machine learning application of it. However, should the reader be interested in the Gini coefficient, we refer to Gini (1). We first show the confusion matrix for a binary classifier in table 3, which is basically a cross-tabulation of the predicted classes vs. the actual classes:

Table 3

	Actual (0)	Actual (1)
Predicted (0)	TN	FN
Predicted (1)	FP	TP

The components are **TN**, the true negatives, i.e., the correctly predicted negatives or class 0, whereas **FN** are the false negatives, i.e., the actual class 1s that are predicted as 0. Similarly, **TP** and **FP** are the true positives and false positives, respectively. Using the entries from the confusion matrix, we have the true positive (**TPR**) and false positive rates (**FPR**),

$$TPR = \frac{TP}{TP + FN}$$
$$FPR = \frac{FP}{FP + TN}$$

Keep in mind that there is a threshold of probability that assigns a prediction to a class. As an example, should 50% be the threshold, any prediction with over 50% will be assigned class 1. Thus, **FPR** and **TPR** are sensitive to the threshold.

Figure 5: ROC Curve for Different Models



Source: CMG Lee, MartinThoma, <https://commons.wikimedia.org/w/index.php?curid=109730045>

The ROC is a plot of **TPR** against **FPR** for different thresholds and is shown in figure 5. Each line or series in the chart represents a model. The dashed red line is that of a model that is a random guess, whereas the blue line is that of an ideal model that predicts each class nearly perfectly, i.e., the off-diagonal elements of the confusion matrix are close to zero. We therefore want a model as close to the blue curve as possible. This is why the green curve represents a model that is better than the model represented by the orange curve. ROC enables us to visually see which model is better. However, a quantity that can be used to compare models or determine performance is the AUC.

7. Area Under the Curve (AUC)

The AUC is simply the area between the curve and the x-axis. The higher the AUC, the better the model at differentiating classes. You may still come across the Gini coefficient being used to measure a model's performance. However, keep in mind that the AUC and Gini are linearly related and synonymous since the relation established by Schechtman and Schechtman (2) is,

$$Gini = 2 * AUC - 1.$$

For a more detailed explanation of the ROC curve with an example, watch the following video:

08:44

[Access video transcript here](#)

8. Conclusion

In this lesson, we looked at tree-based algorithms in classification and regression problems. We examined the methodology behind them and looked at boosting, which combines the power of many weak learners. A modified gradient boosting tree-based algorithm was also discussed as a metric for assessing model performance. In the next lesson, we apply tree-based algorithms to a financial problem.

References

- Gini, Corrado. "On the Measurement of Concentration and Variability of Characters." *Metron - International Journal of Statistics*, vol. 63, no. 1, 2005, pp. 1–38, <https://econpapers.repec.org/article/mtnancoec/0501.htm>. Originally published in Italian in *Atti del Reale Istituto Veneto di Scienze, Lettere ed Arti*, 1914.
- Hull, Isaiah. *Machine Learning for Economics and Finance in TensorFlow 2*. Apress, 2021. https://link.springer.com/chapter/10.1007/978-1-4842-6373-0_4.
- Schechtman, Edna, and Gideon Schechtman. (2016). "The Relationship between Gini Methodology and the ROC Curve." *METRON*, vol. 77, 2019, pp. 171–178, https://www.weizmann.ac.il/math/gideon/sites/math.gideon/files/uploads/AUC_Metron%20May%202019%20v4.pdf.

MACHINE LEARNING IN FINANCE

MODULE 3 | LESSON 4

TREES IN PRACTICE

Reading Time	12 minutes
Prior Knowledge	Tree-based machine learning methodology
Keywords	Model fitting, Training models, Testing models, Model performance, ROC, Area Under the Curve

In the previous lesson, we looked at how tree-based algorithms can be used to predict class outcomes. We now apply this theory to predict one-month returns of a group of stocks exceeding the median or not.

1. Trees in Practice

Recall from Module 3, Lesson 3 that given a set of data with predictors and a target variable with class outcomes, we can fit a decision tree classifier that can intuitively show the rules or criteria that makes a target class highly likely. This non-black box characteristic is a benefit of decision trees as we will show in this lesson by fitting a tree to stock data in order to predict whether a future one-month return will change by more than a threshold or not, i.e., we consider the same binary class problem in lesson 2 of this module.

We therefore use the same data (from Coqueret and Guida chapter 9, section 9.4), specifically the future one-month returns data for a collection of stocks and financial data.

2. The Data

The data can be accessed via

<https://github.com/shokru/mlfactor.github.io/tree/master/material>. Recall that the features we look into are:

- *Mkt_Cap_12M_Usd* - Size
- *Pb* - Price to book
- *Vol1Y_Usd* - For share turnover
- *Return_On_Capital*

- *Roe* - Return on equity
- *Pe* - Price to Earnings ratio

We use Python to perform the analysis and take the same data from the year 2000 to 2018 with only 50 stock IDs. Let us read in the data as in Lesson 2 and take a preview.

```
In [1]: import numpy as np
import pandas as pd

# location of the data

# ENTER LOCATION OF DATA FILE BELOW
df_50 = pd.read_csv("mlfac_dat.csv")

# consider data from 2000 to 2018

df_50 = df_50[(df_50["date"] > "1999-12-31") & (df_50["date"] < "2019-01-01")]

# Filter on the first 50 stock ids

df_50 = df_50[df_50["stock_id"] <= 50]

# Created a class variable for 1-Month returns if
# changes by more than a threshold = thresh

thresh = 0.04
df_50["R1M_Usd_C"] = np.where(df_50["R1M_Usd"].abs() > thresh, 1, 0)

# Make the response variable into integer format

df_50["R1M_Usd_C"] = df_50["R1M_Usd_C"].astype(int)

# Features of interest
feature_cols = ["Mkt_Cap_12M_Usd", "Pb", "Vol1Y_Usd", "Return_On_Capital", "Roe", "Pe"]

df_50 = df_50[feature_cols + ["R1M_Usd_C"]]

df_50.head()
```

```
Out[1]:
```

	Mkt_Cap_12M_Usd	Pb	Vol1Y_Usd	Return_On_Capital	Roe	Pe	R1M_Usd_C
0	0.52	0.21	0.77	0.08	0.02	0.5	0
1	0.52	0.21	0.73	0.08	0.02	0.5	1
2	0.51	0.15	0.67	0.10	0.06	0.5	0
3	0.51	0.15	0.63	0.10	0.06	0.5	1
4	0.52	0.15	0.54	0.10	0.06	0.5	1

We see 6 predictors with the target variable being the one-month future returns (R1M_Usd_C), that is a binary variable taking on 1 if the return is more than 4%

in either a long or short direction and 0 otherwise. Keep in mind that features have been pre-selected in this example. In practice, however, feature selection generally takes a significant time to perform and often adds much value in improving performance.

3. Train the Tree

Now that we have a view of the data, we will proceed with splitting the dataset into a train/test 80/20 split. All rights reserved WQU WorldQuant University QQQQ

```
In [2]: from sklearn.model_selection import train_test_split

# split dataset in features and target variable
X = df_50[feature_cols] # Transformed Features

y = df_50["R1M_Usd_C"] # Target variable

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=1)
```

We now declare our decision tree classifier using the `sklearn` package.

```
In [3]: # Import Decision Tree Classifier

from sklearn.tree import DecisionTreeClassifier

# Create Decision Tree classifier object

clf = DecisionTreeClassifier(criterion="gini", max_depth=3,
min_samples_leaf=25)

# Train Decision Tree classifier

clf = clf.fit(X_train, y_train)
```

The decision tree produces a set of rules that are easily interpretable in our case since we have used only a few features for illustrative purposes. A flow diagram of the decision tree can be shown using the code below, which will produce a flow diagram as in Figure 1. This shows that companies with turnover above 0.905 are 78% likely to have returns that change by more than 4%. This is a strong differentiating node.

```
In [4]: # visualize the tree
```

```
import matplotlib.pyplot as plt
from sklearn import tree

# features we used

fn = feature_cols

# labels of the target class

cn = ["0", "1"]

fig = plt.figure(figsize=(25, 20))
_ = tree.plot_tree(clf, feature_names=fn, class_names=cn, filled=True)

# save the figure to file

# fig.savefig('imagename.png')
fig.savefig("decision_tree.png")
```

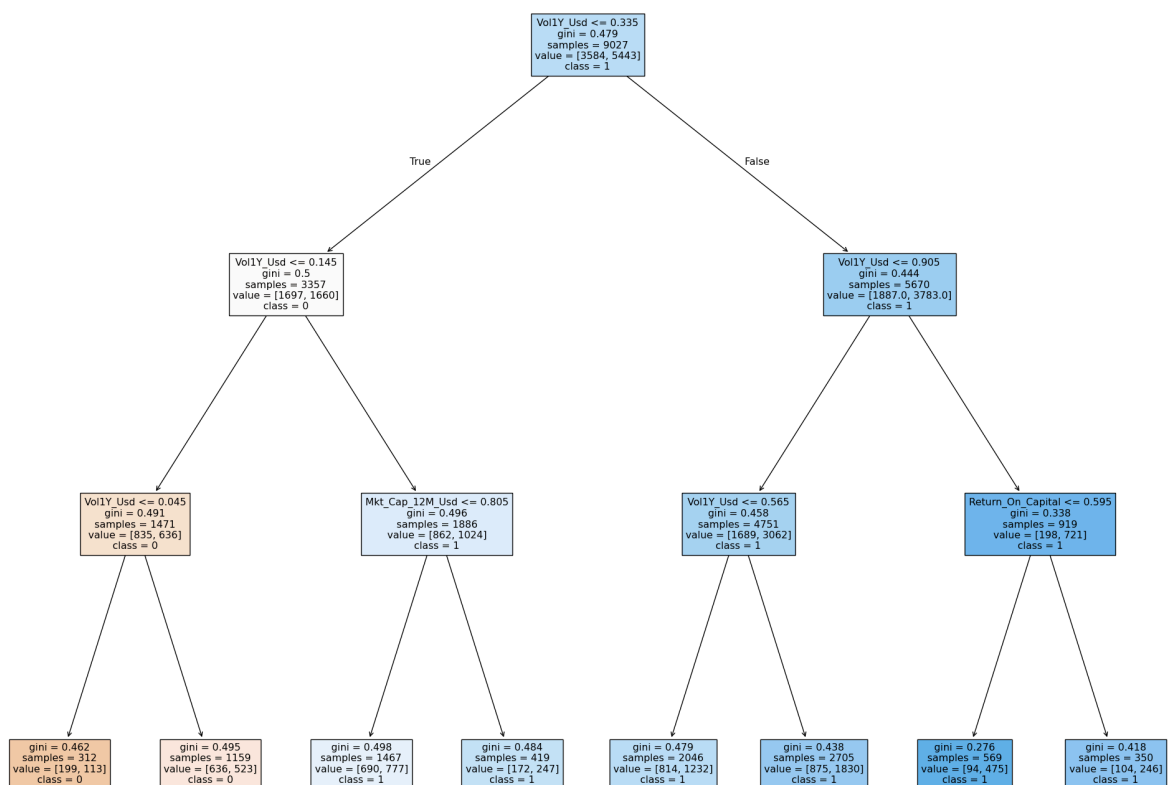


Fig. 1: Flow Diagram of Decision Tree for Predicting Price Change of More than 4% in Any Direction

Figure 1 shows an absolute change exceeding 4% as class 1 and class 0 otherwise. The blue nodes represent cases that majority have class 1, i.e., the darker the shade of blue, the purer the node. Similarly, for Class 0, the more orange the node, the higher proportion of Class 0.

It is worth pointing out that the cuts in Trees are parallel to existing axis whereas in SVM the cuts can be, but not restricted to, a linear combination of features. The sequential nature of the splitting shown in Fig 1 shows that only one feature can be used at a time. SVM on the other hand is not restricted by this.

4. Tree Performance

We've trained our decision tree classifier and we are now ready to test the performance. To do this, we need to run the unseen test dataset through the classifier to determine the class predictions for the target variable. It would be insightful to compare this to a random guess classifier, i.e., a model that requires no skill.

```
In [5]: # Predict the response for test dataset

y_pred = clf.predict(X_test)

# Obtain the probabilities
probs_tmp = clf.predict_proba(X_test)

probs = probs_tmp[:, 1]
```

Fortunately, `Sklearn` has tools to evaluate performance metrics for most machine learning algorithms. We look at the ROC curve and AUC as in Lesson 2.

```

In [6]: import matplotlib.pyplot as plt
        from sklearn.metrics import roc_auc_score, roc_curve

        # plot the roc curve for the model

        # generate a no skill or random guess prediction

        ns_probs = [0 for _ in range(len(y_test))]

        # calculate roc curves for random guess

        ns_fpr, ns_tpr, _ = roc_curve(y_test, ns_probs)

        # calculate roc curves for the decision tree

        fpr, tpr, _ = roc_curve(y_test, probs)

        # calculate scores

        ns_auc = roc_auc_score(y_test, ns_probs) # random guess
        tree_auc = roc_auc_score(y_test, probs) # tree classifier

        # summarize scores

        print("No Skill: ROC AUC=%.3f" % (ns_auc))

        print("Classifier: ROC AUC=%.3f" % (tree_auc))

        plt.plot(ns_fpr, ns_tpr, linestyle="--", label="No Skill")

        plt.plot(fpr, tpr, marker=".", label="Tree Classifier")

        # axis labels

        plt.xlabel("False Positive Rate")

        plt.ylabel("True Positive Rate")

        # show the legend

        plt.legend()

        # show the plot

        plt.show()

```

```

No Skill: ROC AUC=0.500
Classifier: ROC AUC=0.606

```

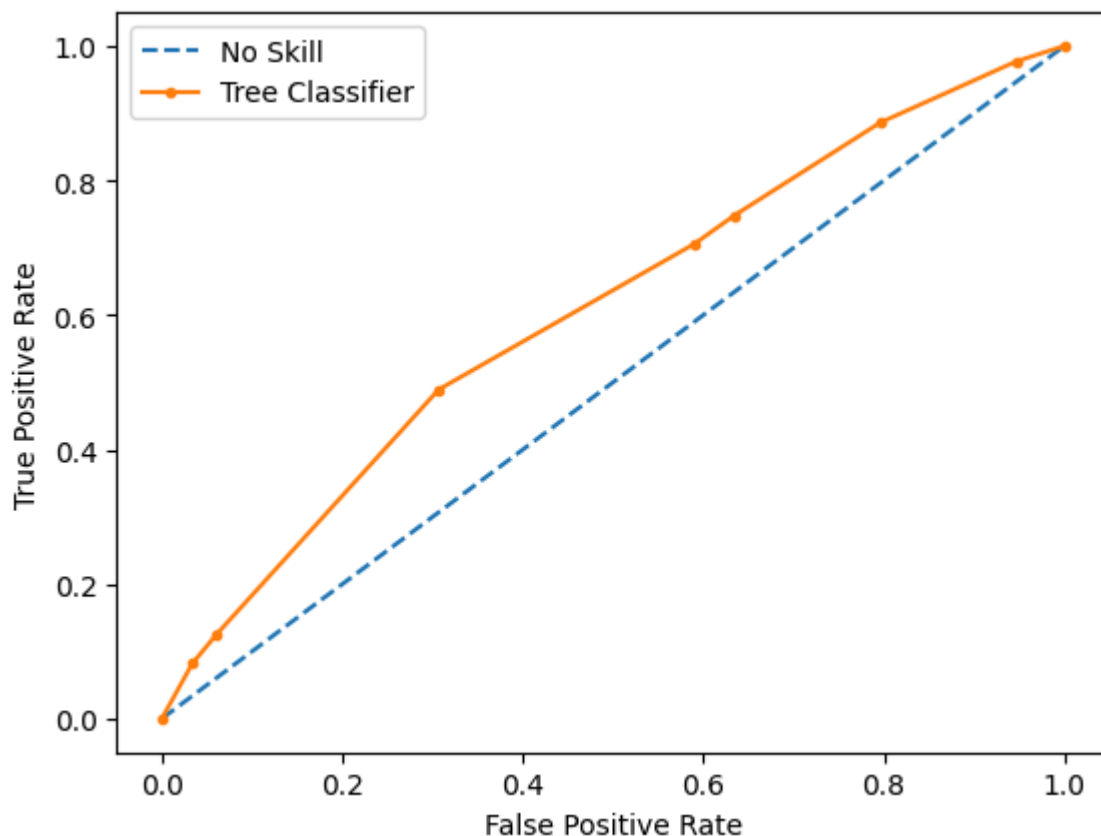


Fig. 2: ROC Curve of the Decision Tree Classifier vs. a Random Guess or No Skill Predictor

The AUC for the classifier is 60.6%, which is better than a random guess. It is worth noting that a subset of predictors was chosen to illustrate the decision tree flow diagram in Figure 1. Feature selection plays an important part in model performance and should not be overlooked.

Lastly, it is worth looking at the implementation of a tree-based algorithm that uses boosting. The algorithm we consider is popular in the Kaggle community and can be easily implemented in Python using the `xgboost` package. The hyperparameters have been specified, but not optimized, instead of the default settings. For further reading on these parameters, please refer to ("[XGBoost Parameters — Xgboost 1.6.1 Documentation](#)")

```
In [7]: !pip install xgboost
```

Requirement already satisfied: xgboost in /usr/local/lib/python3.11/site-packages (3.2.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.11/site-packages (from xgboost) (2.1.3)

Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.11/site-packages (from xgboost) (2.30.7)

Requirement already satisfied: scipy in /usr/local/lib/python3.11/site-packages (from xgboost) (1.15.1)

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>. Use the --root-user-action option if you know what you are doing and want to suppress this warning.

[notice] A new release of pip is available: 25.0.1 -> 26.1.2

[notice] To update, run: `pip install --upgrade pip`

In [8]: `import xgboost as xgb`

```
# Init classifier
xgb_cl = xgb.XGBClassifier(
    max_depth=2,
    eta=0.1, # learning rate
    objective="binary:logistic",
    eval_metric="auc",
)

# Fit
xgb_cl.fit(X_train, y_train)

# Predict
preds_xgb = xgb_cl.predict(X_test)

# Obtain the probabilities
preds_prob_xgb_tmp = xgb_cl.predict_proba(X_test)

preds_prob_xgb = preds_prob_xgb_tmp[:, 1]
```

The results of the `xgboost` model along with the decision tree and random guess classifier is shown in figure 3 below. The `XGBoost` ties with the decision tree at 60.6%. The reader, as an exercise, can decrease the max depth of the tree to determine the change in performance. This will show the sensitivity to the hyperparameters.

```

In [9]: # generate a no skill or random guess prediction
ns_probs = [0 for _ in range(len(y_test))]
# calculate roc curves
# random
ns_fpr, ns_tpr, _ = roc_curve(y_test, ns_probs)
# tree
fpr, tpr, _ = roc_curve(y_test, probs)
# xgb
xgb_fpr, xgb_tpr, _ = roc_curve(y_test, preds_prob_xgb)

# calculate scores
ns_auc = roc_auc_score(y_test, ns_probs) # random guess
tree_auc = roc_auc_score(y_test, probs) # tree classifier
xgb_auc = roc_auc_score(y_test, preds_prob_xgb) # tree classifier
# summarize scores
print("No Skill: ROC AUC=%.3f" % (ns_auc))
print("Tree Clasifier: ROC AUC=%.3f" % (tree_auc))
print("XGB Clasifier: ROC AUC=%.3f" % (xgb_auc))

plt.plot(ns_fpr, ns_tpr, linestyle="--", label="No Skill")
plt.plot(fpr, tpr, marker=".", label="Tree Classifier")
plt.plot(xgb_fpr, xgb_tpr, marker=".", label="XGB Classifier")
# axis labels
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
# show the legend
plt.legend()
# show the plot
plt.show()

```

No Skill: ROC AUC=0.500
 Tree Clasifier: ROC AUC=0.606
 XGB Clasifier: ROC AUC=0.606

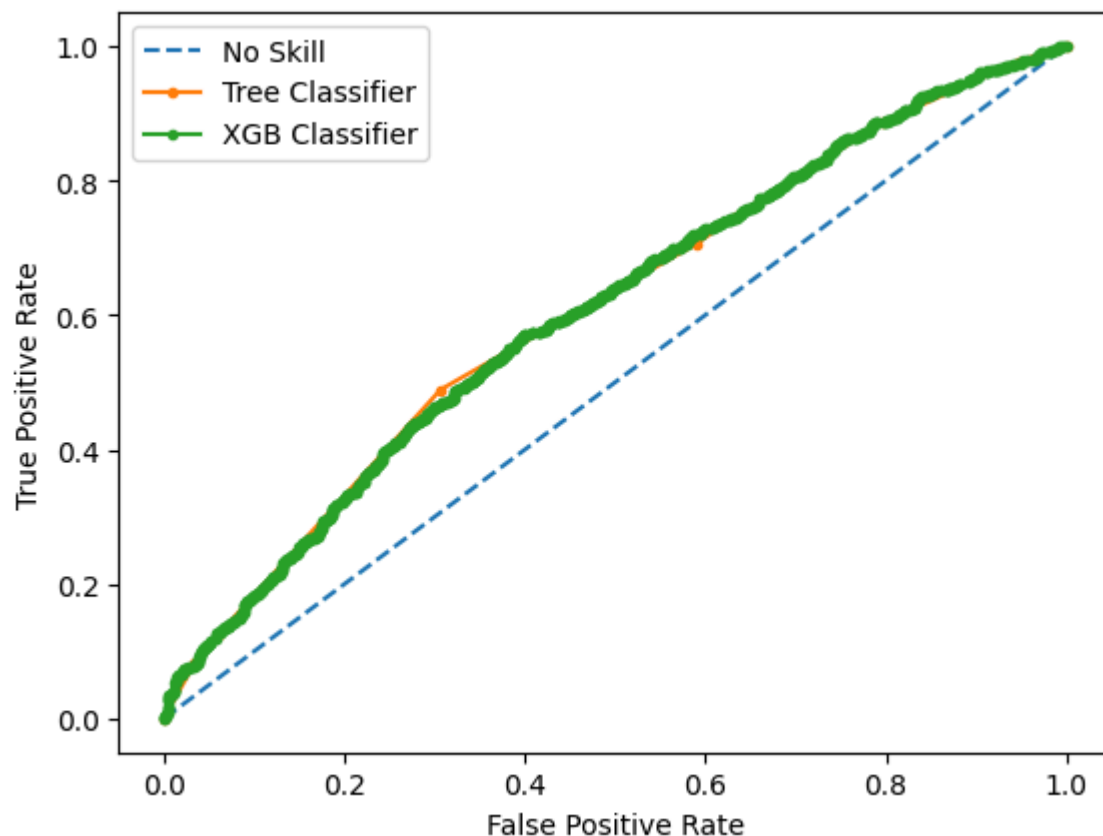


Fig. 3: ROC Curves for the Tree, XGBoost, and No-Skill Model

5. Conclusion

The examples illustrated in this lesson help us understand the tree-based algorithms from the simple decision tree to the xgboost model. The model performance is better than a random guess thus showing added benefit. This lesson focused on showing how to apply tree-based algorithms to a real-world classification problem. A model that does better than a random guess can be easily motivated; however, keep in mind the goal of the specific classification problem, as it may require a minimum AUC to be achieved. The next module will explore support vector machines and neural networks, another class of predictive models.

References

- Coqueret, Guillaume, and Tony Guida. *Machine Learning for Factor Investing: R Version*. CRC Financial Mathematics Series, Chapman and Hall/CRC, 2022.
- "XGBoost Parameters — Xgboost 1.6.1 Documentation." XGBoost Parameters, <https://xgboost.readthedocs.io/en/stable/parameter.html>. Accessed 30 June 2022.